

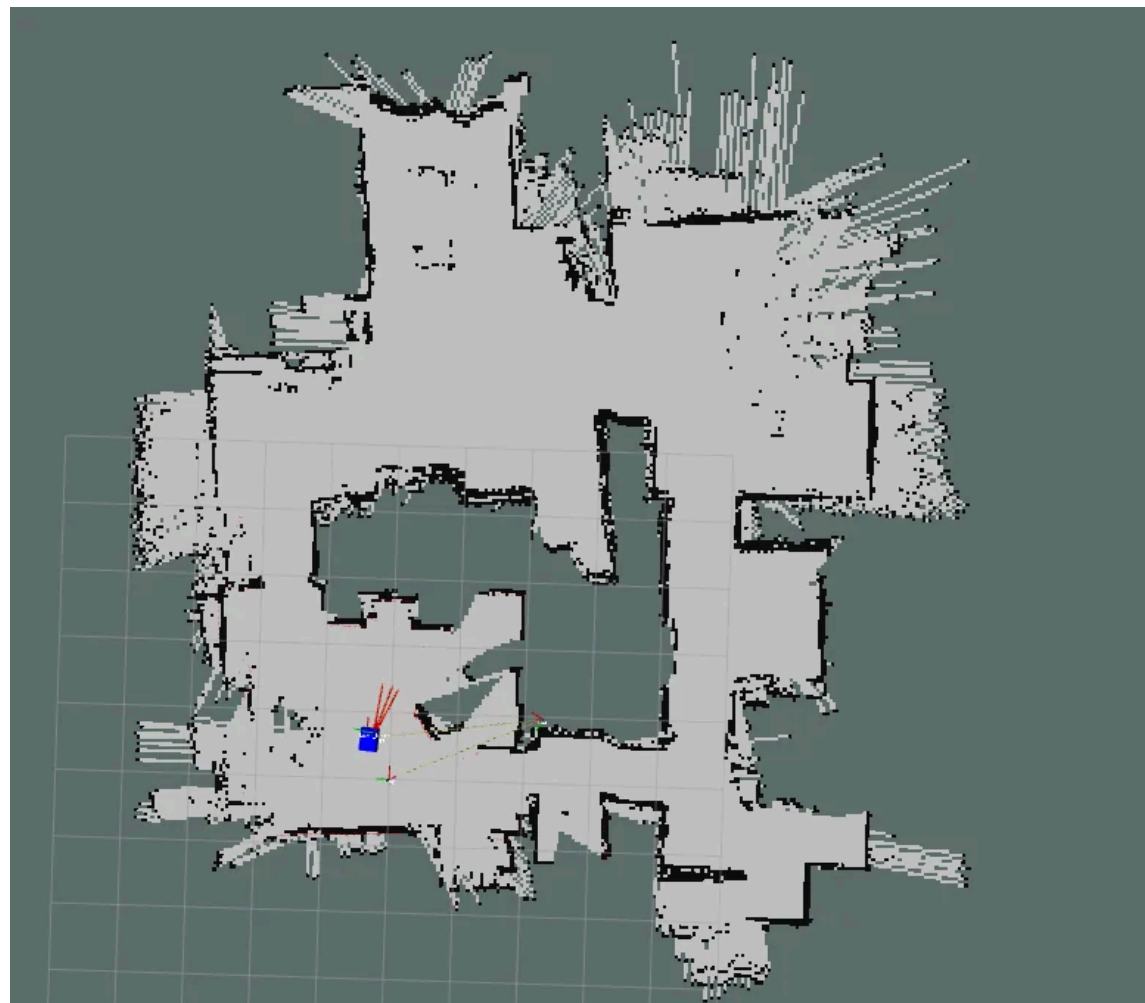
SLAM 기초

문제 정의 · 분류 · SLAM의 수학적 기초 · Front-end와 Back-end · Loop Closure

cchyun@gmail.com

Part 1. SLAM 문제 정의 & 분류

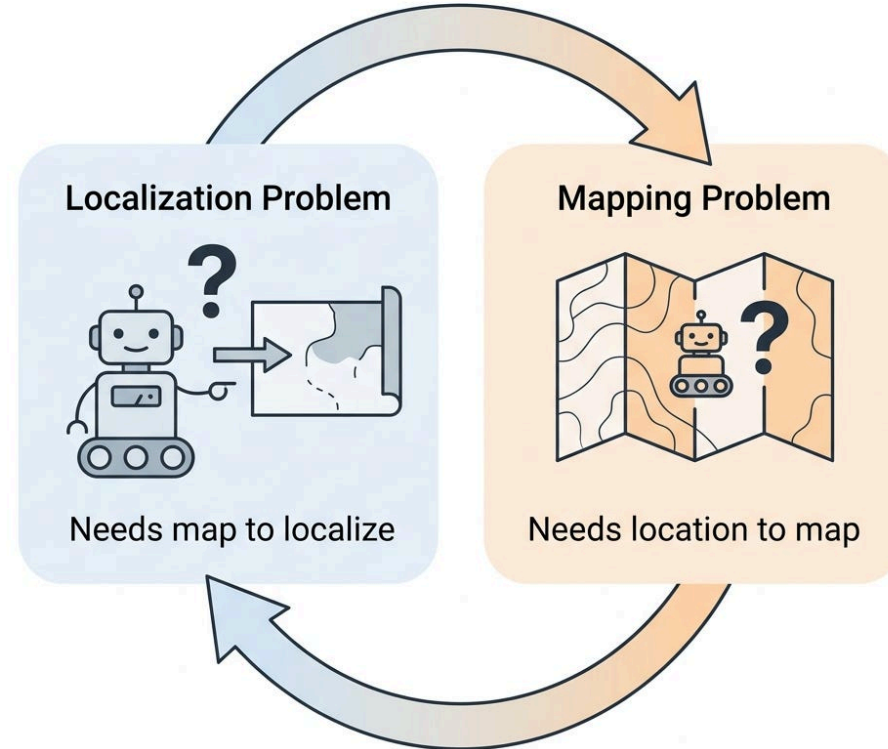
- 실내·지하 환경에서는 GPS 신호가 닿지 않아 로봇 스스로 위치를 파악해야 함
 - Scenario A — 지도 없이 주행할 때
 - TurtleBot3가 바퀴 회전수(Odometry)만 믿고 복도를 이동
 - 바퀴 미끄러짐 + 센서 오차가 누적 → 로봇은 직진한다고 믿지만 실제로는 벽에 부딪힘
 - Scenario B — SLAM이 있을 때
 - LiDAR(레이저 센서)로 벽까지의 거리를 측정하며 이동
 - 자신의 위치를 보정하며 실시간으로 지도를 생성
 - "내가 지금 복도의 3번째 문 앞에 있구나" → 정확한 자기 위치 인식 가능



SLAM의 핵심 문제 — "닭이 먼저냐 달걀이 먼저냐"

- 위치 추정과 지도 작성이 동시에 일어나야 한다는 순환 딜레마
 - 지도가 있어야 위치를 안다
 - 현재 센서에 보이는 벽 모양을 이미 그려진 지도와 비교해야 위치를 알 수 있음
 - 위치를 알아야 지도를 그린다
 - 센서로 측정한 벽의 위치를 지도 위에 찍으려면 로봇의 좌표(x, y)를 정확히 알아야 함
- SLAM의 정의
 - Simultaneous Localization And Mapping
 - 동시적 위치 추정 및 지도 작성
 - "지도 없이 위치 모르고, 위치 모르면 지도 못 만드는데" 순환 문제를 해결하는 과정

The 'Chicken and Egg' Problem in SLAM



- 정의: Simultaneous Localization And Mapping
 - 로봇이 미지의 환경에서 자신의 Trajectory를 추정하고 일관된 Map을 동시에 구축하는 과제
- 핵심 딜레마 — "닭이 먼저인가, 달걀이 먼저인가"
 - 정확한 Mapping에는 정밀한 위치 정보가 필요함
 - 신뢰할 수 있는 Localization에는 기존 Map이 필요함
 - 두 문제를 동시에 해결해야 하는 Dual estimation 문제

- 잡음이 섞인 센서 측정값을 Motion 예측과 재귀적으로 융합

- 로봇의 센서(라이다, 카메라, IMU, 휠 오도메트리 등)는 물리적 한계와 환경적 요인으로 인해 항상 잡음(noise)이 포함
- 결정론적(deterministic) 접근은 이런 잡음을 무시하므로, 작은 오차도 누적되어 급격히 발산
- 확률론적 접근은 불확실성을 명시적으로 모델링하여 점진적으로 정제
- Agent의 이동 경로($x_{1:t}$)와 Map 구조(m)에 대한 **결합 사후 확률(Joint posterior)**에 수렴
 - $z_{1:t}$: 시간 1부터 t 까지의 **센서 측정값(observation)**
 - $u_{1:t}$: 시간 1부터 t 까지의 **제어 입력(control input)** (모터 명령, 속도 등)

$$p(x_{1:t}, m \mid z_{1:t}, u_{1:t})$$

- 알고리즘 발전 과정

- 초기: 필터링 기반 (Extended Kalman Filter, EKF)
- 현대: **Graph-based SLAM**을 활용하는 최적화 기반 아키텍처

- **센서 데이터의 차원에 따른 분류**
 - **2D SLAM**
 - 로봇이 평면(지면) 위에서만 움직인다고 가정
 - 주로 **바퀴 달린 로봇(TurtleBot3, 로봇 청소기)**에 사용
 - 연산량이 적어 저사양 컴퓨터에서도 잘 작동
 - **3D SLAM**
 - 로봇의 높낮이와 기울기까지 고려
 - **드론**처럼 입체적인 움직임이 필요한 분야에 필수
 - 데이터가 방대해 높은 사양의 컴퓨팅 능력이 필요

- 처리 시점에 따른 분류

- 온라인(Online) SLAM

- 현재의 로봇 위치와 방금 찍은 지도를 **실시간으로 업데이트**하는 데 집중
- 필터 기반 방식이 여기에 해당
- 실시간 주행**에 유리

$$p(x_t, m \mid z_{1:t}, u_{1:t})$$

- 풀(Full) SLAM

- 로봇이 처음부터 끝까지 이동한 전체 경로와 전체 데이터를 **한꺼번에 최적화**
- 가장 정확한 지도를 만들 수 있음
- 주로 주행을 마친 후 **정밀한 지도 생성** 시 사용

$$p(x_{1:t}, m \mid z_{1:t}, u_{1:t})$$

- 알고리즘 방식에 따른 분류

- 필터 기반 (EKF-SLAM, 파티클 필터)

- 매 순간 새로운 센서 값이 들어올 때마다 **확률적으로 위치를 업데이트**
- 파티클 필터(AMCL)**: 예상 위치에 수많은 '입자'를 뿌려 확률이 높은 곳으로 수렴

- 그래프 기반 (Pose Graph Optimization)

- 로봇의 궤적을 **점(Node)** 으로, 센서 측정을 **선(Edge)** 으로 연결해 그물망(그래프) 구성
- 이후 그물망 전체를 조여 오차를 줄이는 방식
- 현재 ROS2의 SLAM Toolbox** 등 산업계에서 가장 많이 쓰는 방식

- 센서 종류와 모달리티에 따라 SLAM의 정확도와 활용 방식이 달라짐

- LiDAR SLAM

- 레이저를 360도 회전시키며 주변 장애물과의 거리를 측정
- 빛을 쏘아 돌아오는 시간으로 계산 → **매우 정밀**
- 주로 2D LiDAR (TurtleBot3 기본 장착) 에 사용

- Visual SLAM

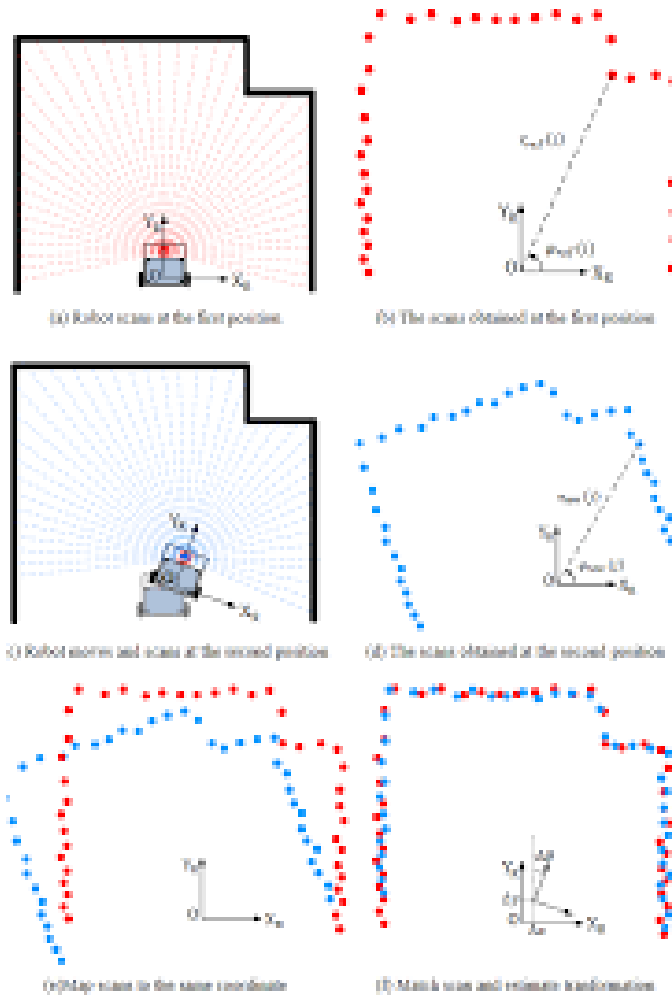
- 일반 컬러 영상(RGB)을 처리하여 기하학적 제약을 추출
- RGB-D 카메라(RGB + Depth)는 색상·질감 정보가 많아 **특정 문이나 가구 같은 랜드마크 인식에 유리**

- 다중 센서 융합 (LiDAR-IMU 등)

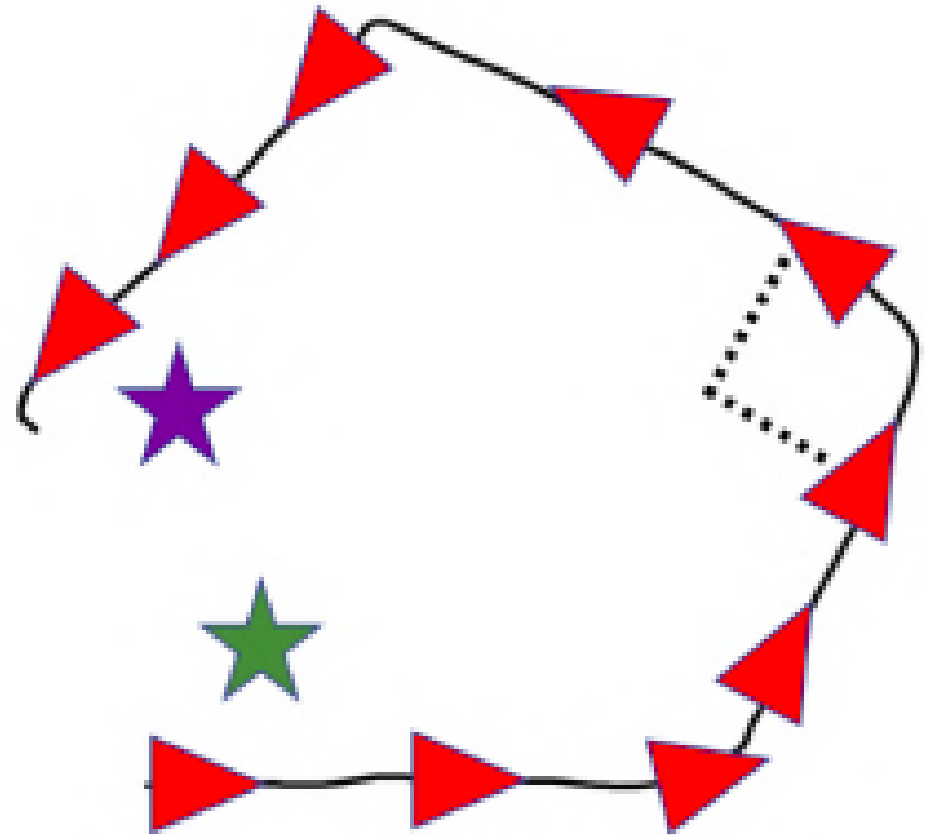
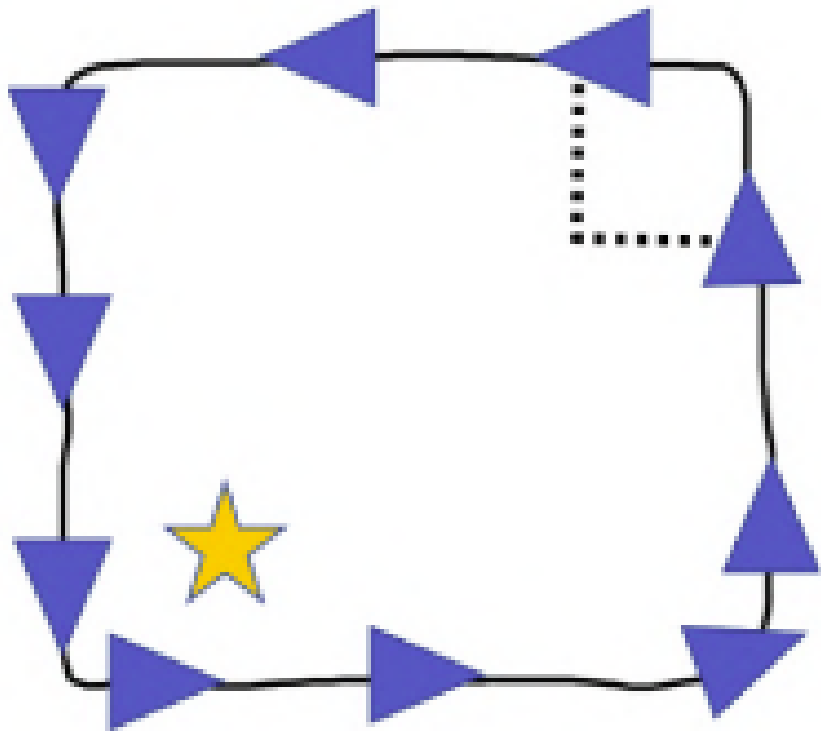
- 단일 센서의 한계를 상호 보완 → 견고성(robustness) 극대화
- 가속도와 회전 각속도를 재는 IMU를 LiDAR나 카메라와 함께 사용
- 바퀴가 헛돌아도 **실제 기울기·회전을 보완** → 더 정확한 위치 추정

- 바퀴 회전수만으로 위치를 추정하면 오차가 점점 쌓임
 - 오차의 원인 — 드리프트(Drift)
 - 바퀴의 마찰력 차이, 울퉁불퉁한 바닥
 - 인코더 센서의 미세한 오차가 시간이 지날수록 크게 누적
 - SLAM의 해결 방법 — Scan Matching
 - 로봇은 바퀴만 믿지 않음
 - "바퀴는 1m 갔다고 하지만, LiDAR로 보니 벽 모양이 그대로인걸?"
 - "그럼 실제로는 0.9m만 간 거구나" → 센서 데이터 비교로 오차를 지속적으로 교정

Scan Matching



- 로봇이 이전에 방문한 장소를 알아보는 기능
 - 발생 상황
 - 로봇이 복도를 한 바퀴 돌아 다시 출발점에 왔을 때
 - 쌓인 오차 때문에 지도가 어긋나 보이는 현상 발생
 - 동작 원리
 - 로봇이 "어? 여기 아까 왔던 곳인데!"라고 **이전 방문 장소를 재인식**
 - 장소를 재인식하는 순간, 쌓였던 모든 위치 오차를 **한꺼번에 교정**하여 지도를 정렬
 - 중요성
 - Loop Closure 기능이 없으면 지도는 계속 꼬임
 - 좋은 지도를 위해 **왔던 길을 가끔 다시 돌아가며(Loop 유도) 주행**하는 것이 중요



Part 2. SLAM의 수학적 기초

- 목표

- 센서 측정값 $z_{1:t}$ 와 제어 입력 $u_{1:t}$ 가 주어졌을 때, 로봇의 이동경로 $x_{1:t}$ 와 Map m 에 대한 Posterior 추정

- Joint posterior

- $p(x_{1:t}, m \mid z_{1:t}, u_{1:t})$
- Given
 - $z_{1:t}$: 시간 1부터 t 까지의 센서 측정값 (Observation)
 - $u_{1:t}$: 시간 1부터 t 까지의 제어 입력 (Control input)
- Wanted
 - $x_{1:t}$: 시간 1부터 t 까지의 로봇 이동 경로 (Trajectory)
 - m : 구축하고자 하는 지도 (Map)

- 두 가지 핵심 확률 모델

- Motion Model: $p(x_t | x_{t-1}, u_t)$

- 제어 입력 u_t 로 상태를 **예측** — 불확실성 증가

- Measurement Model: $p(z_t | x_t, m)$

- 실제 측정 z_t 로 추정 (x_t, m) 를 **평가·보정** — 불확실성 감소

- SLAM의 본질

- Motion으로 **예측** → Measurement로 **보정**을 반복
 - 잡음이 섞인 현실에서 점진적으로 수렴

- 정의

- 제어 입력에 따라 로봇의 Pose가 시간에 따라 어떻게 변화하는지 설명

- 확률 형태

- $p(x_t \mid x_{t-1}, u_t)$

- 예시: 2D 차동 구동 속도 기반(Velocity-based) 모델

- Pose: $x_t = [x, y, \theta]^T$
- 제어 입력(속도): 선속도 v , 각속도 ω
- 누적 모션 계산:

$$x_t = x_{t-1} + v \cos \left(\theta_{t-1} + \frac{\omega \Delta t}{2} \right) \Delta t$$

$$y_t = y_{t-1} + v \sin \left(\theta_{t-1} + \frac{\omega \Delta t}{2} \right) \Delta t$$

$$\theta_t = \theta_{t-1} + \omega \Delta t$$

(단, Δt 는 제어 주기 또는 시간 간격을 의미)

- 정의
 - 로봇의 Pose, Map, 센서 측정값 간의 관계를 설명
- 확률 형태
 - $p(z_t \mid x_t, m)$
- 예시
 - 2D LiDAR: 알려진 Landmark까지의 거리와 방위각 예측

- 핵심 아이디어

- SLAM에서 Pose에 대한 확률적 믿음(확률 분포)을 나타냄

$$bel(x_t) = p(x_t | z_{1:t}, u_{1:t})$$

- 시각 $t - 1$ 의 Belief $bel(x_{t-1})$ 으로부터 시각 t 의 Belief $bel(x_t)$ 를 재귀적으로 계산

- ① Prediction 단계 (Time update / Motion update)

- Motion model을 사용하여 Belief를 시간 전방으로 투영
- 로봇 Motion의 불확실성을 전파
→ **Predicted Belief**는 본질적으로 더 불확실함

$$\overline{bel}(x_t) = \int p(x_t | x_{t-1}, u_t) bel(x_{t-1}) dx_{t-1}$$

- ② Update 단계 (Measurement update / Correction step)

- 획득된 센서 측정값 z_t 를 사용하여 예측된 Belief를 정제

- 갱신 규칙 (Bayes' theorem 유도)

- η : Normalization constant

$$bel(x_t) = \eta \cdot p(z_t | x_t) \overline{bel}(x_t)$$

- 효과

- 센서 측정값의 정보가 풍부하다면(잡음이 낮다면), Likelihood $p(z_t | x_t)$ 가 높은 상태에 **확률 질량을 집중**
- Belief 분포의 **평균이 측정과 일치하는 방향으로 이동**, 분산이 감소하여 불확실성 감소
- 예측-갱신 주기의 재귀적 특성이 실시간 상태 추정의 중추

- ① Prediction (예측): "눈 감고 걸어가기"
 - 로봇이 바퀴 제어 명령(u_t)에 따라 이동할 때
 - 바퀴의 미끄러짐, 바닥의 요철 등으로 오차가 누적됨
 - **결과: 로봇 위치에 대한 확률 분포 $bel(x)$ 가 넓게 퍼지고 흐려짐 (불확실성 증가)**
- ② Update (갱신): "눈 뜨고 주변 확인하기"
 - 로봇이 LiDAR 센서 등으로 벽과의 거리(z_t)를 측정하여 기존 지도와 대조할 때
 - **결과: 확률 분포 $bel(x)$ 가 실제 위치를 중심으로 좁고 뾰족하게 수렴함 (불확실성 감소)**
- 베이저안 필터의 반복
 - 주행(예측 $\rightarrow$ 불확실성 $\uparrow$) $\Rightarrow$ 센서 관측(갱신 $\rightarrow$ 불확실성 $\downarrow$)의 재귀적 반복

- 정의

- 측정 가능도(Likelihood)만을 최대화하여 최적해를 찾음

- 수식

$$(x_{1:t}^*, m^*)_{ML} = \operatorname{argmax}_{x_{1:t}, m} p(z_{1:t} \mid x_{1:t}, m)$$

- Likelihood 항 $p(z_{1:t} \mid x_{1:t}, m)$

- "이 Trajectory와 Map이 참이라면, 실제로 관측된 데이터 $z_{1:t}$ 가 나올 확률"
- Measurement Model의 누적곱 — 예상 측정과 실제 측정의 일치도 평가
- 데이터와의 **적합도(fitness)** 를 측정하는 핵심 항

- 특징

- 그래프 기반 SLAM의 전형적인 예
- Pose graph에 대한 **Non-linear least squares** 최적화를 추구
- **한계**: 잡음이 많은 데이터에서 **Overfitting** 위험

- 정의

- ML에 **Prior(사전 지식)** 를 추가하여 더 robust한 추정

- 수식

$$(x_{1:t}^*, m^*) = \operatorname{argmax}_{x_{1:t}, m} \left[p(z_{1:t} \mid x_{1:t}, m) \cdot p(x_{1:t}, m) \right]$$

- Prior 항** $p(x_{1:t}, m)$

- MAP 추정이 해를 Regularization하여 잡음이 많은 데이터에 대한 Overfitting을 방지
- 직관적 예시**
 - "로봇이 순간이동하지는 않는다" → 연속적인 Trajectory 선호
 - "벽은 갑자기 사라지지 않는다" → 일관된 Map 선호

- ML과의 관계

- Prior를 **Uniform(상수)** 로 가정하면 ML과 동일
- ML은 MAP의 특수 경우

Part 3. Graph SLAM 아키텍처

Front-end와 Back-end

- **역할**

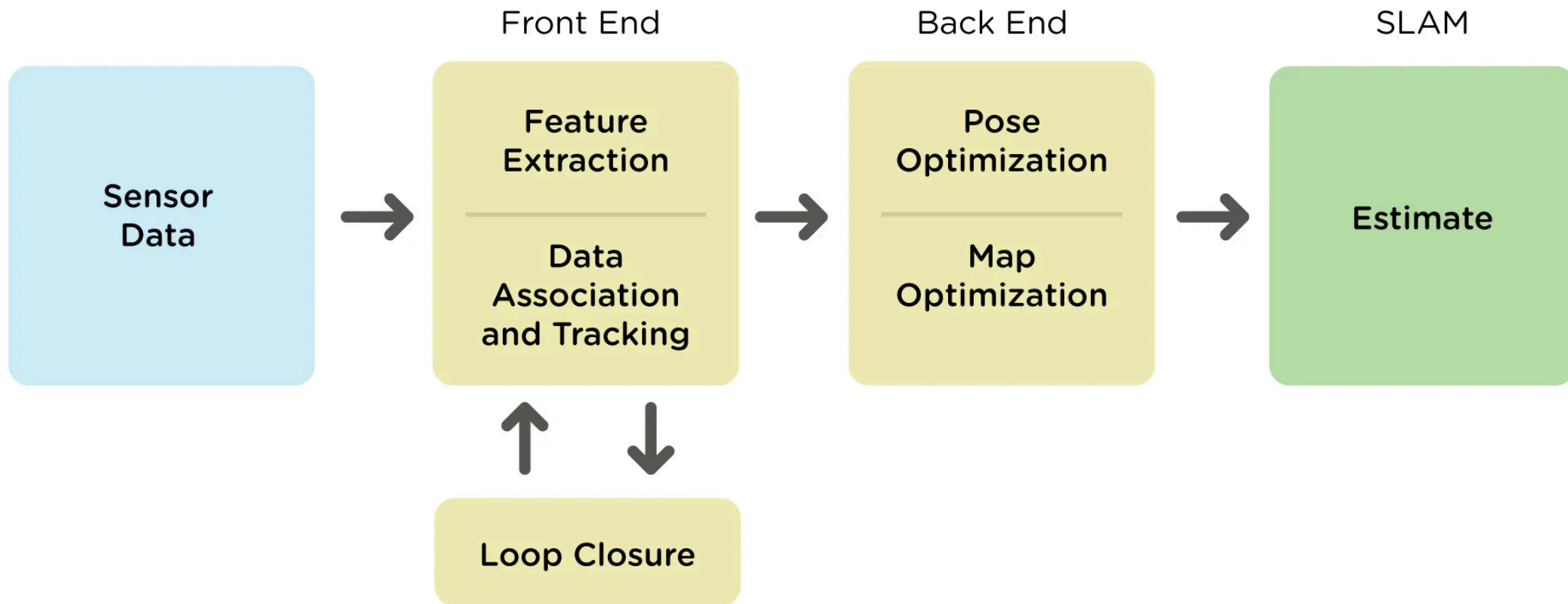
- 원시 센서 데이터 스트림에서 의미 있는 기하학적 정보(Pose, 특징점, 구조 등)를 추출
- 상대적 Pose 제약 조건(Odometry 또는 Loop closure factor)으로 변환

- **핵심 과제: Data association**

- 서로 다른 시점 간에 동일한 물리적 특징에 해당하는 관찰을 일치시킴

- **Outlier 방어**

- 잡음이 많은 데이터에서 Outlier를 걸러내고 Inlier만을 Back-end로 전달
- 추정(Belief)과 측정의 불확실성(공분산)으로 예측-측정 불일치 정도를 통계적으로 판단
- 잘못된 Data association이 Back-end를 손상시키지 않도록 방어



- Front-end의 주요 기능: Odometry

- Visual Odometry 또는 LiDAR Scan matching
- 연속적인 프레임 간의 상대 변환(회전 및 변환)을 추정

- Dead reckoning

- 추정값을 시간에 따라 통합하는 과정

$$x_t = x_{t-1} \oplus u_t$$

- Drift(표류)

- 센서 잡음으로 인해 오차가 시간이 지남에 따라 누적되는 현상
- 무한정 커지므로 전역 최적화와 Loop closure가 필수적

- 역할
 - Front-end로부터 제약 조건을 받아 전체 경로와 Map에서 최적 해를 찾는, 전역 일관성을 복원하는 최적화 엔진
- Pose Graph Optimization
 - 각 Node: 로봇의 Pose(키프레임)
 - 각 Edge: 상대 Pose 제약 조건(Odometry, Loop closure)
- 최적화 방법
 - Non-linear least squares로 전체 Pose 간 오차를 최소화
 - 잘못 누적된 Drift를 Loop closure 제약으로 전역적으로 교정

- 오차 함수

- 예상 상대 Pose와 측정된 상대 Pose 간의 차이를 측정

$$e_{ij}(x_i, x_j) = \log(z_{ij}^{-1} \cdot (x_i^{-1} x_j))$$

- 총 비용 함수

- 모든 Edge에 대한 Mahalanobis distance의 제곱 합

$$F(X) = \sum_{\langle i,j \rangle \in \mathcal{E}} e_{ij}(x_i, x_j)^T \Omega_{ij} e_{ij}(x_i, x_j)$$

- Information Matrix Ω_{ij}

- 측정의 신뢰도를 정량화하는 **역공분산 행렬**
- 높은 값은 정밀한 측정을 의미
- 신뢰도 높은 Edge들이 먼저 수렴

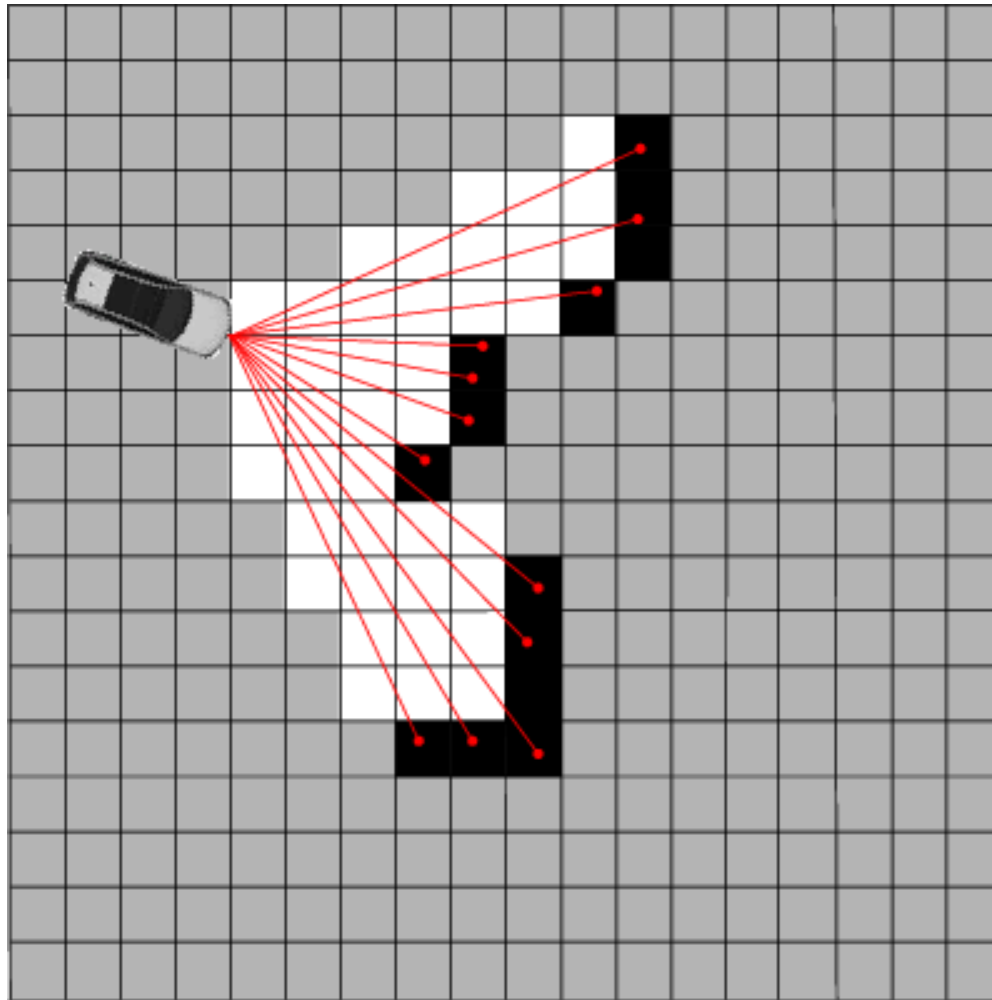
- Pose Graph의 특징
 - 각 Pose는 이웃 Pose 몇 개와만 연결
 - 대부분의 Pose 쌍은 직접 연결되지 않음
- 희소 행렬 (Sparse Matrix)
 - 연결 안 된 Pose 쌍 = 0
 - 1000개 Pose 중 실제 0이 아닌 원소는 1% 미만
- 효과
 - 0은 저장·계산 안 함 → 메모리 절약
 - 희소 행렬 전용 알고리즘으로 실시간 최적화 가능

Part 4. OccupancyGrid 지도 이해

OccupancyGrid 지도란?

- 점유 격자 지도(Occupancy Grid Map, OGM)는 로봇이 환경을 격자(Grid) 형태의 셀(Cell)로 분할하고, 각 셀의 장애물 점유 확률을 표현한 지도
 - ROS 2에서 `nav_msgs/msg/OccupancyGrid` 메시지로 통신
 - 주요 필드 구조:
 - **header**: 데이터 생성 시간과 좌표계 정보(`frame_id: map`)
 - **info (MapMetaData)**: 지도의 메타데이터 (해상도, 크기, 원점)
 - **data**: 실제 점유 확률 데이터가 담긴 1차원 배열

OccupancyGrid 지도란?



■ $p_{z_{k+1}}(O_{k+1} | z_{k+1}) = 0.95$

□ $p_{z_{k+1}}(O_{k+1} | z_{k+1}) = 0.05$

■ $p_{z_{k+1}}(O_{k+1} | z_{k+1}) = 0.5$

- info 필드의 주요 파라미터

- `resolution`: 격자 한 셀의 실제 크기 (단위: m/pixel)
 - 예: `0.05` → 셀 하나 = 실제 5cm
- `width` / `height`: 지도의 가로 및 세로 셀 개수
- `origin`: 지도의 원점(좌측 하단) 좌표 `[x, y, yaw]`
- 해상도와 품질의 트레이드오프:
 - 해상도가 높을수록(값이 작을수록) 더 세밀한 표현 가능
 - 단, 메모리 사용량·연산량 증가 및 노이즈 민감도 상승

- 각 셀은 **-1**에서 **100** 사이의 정수 값을 가짐

셀 값	의미	RViz2 색상
0	자유 (Free) — 장애물 없음, 이동 가능	흰색
100	점유 (Occupied) — 벽·장애물 존재	검은색
-1	미탐색 (Unknown) — 센서 미도달	회색

- **.pgm** 이미지 픽셀 값 (0~255)과 직접 대응
 - 흰색(254~255) → Free / 검은색(0) → Occupied / 회색(~205) → Unknown

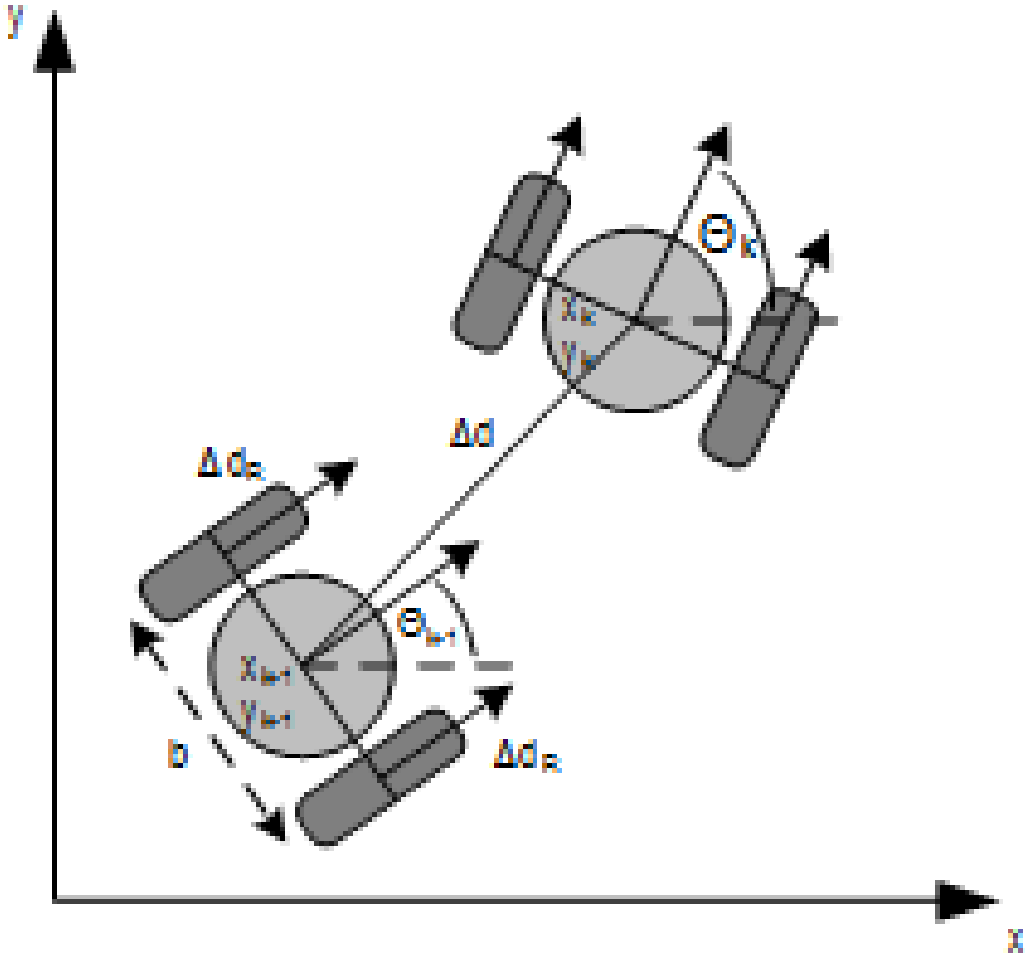
파티클 필터 & MCL 알고리즘

Particle Filter · MCL · AMCL

Part 1. 로컬라이제이션(Localization) 개요

로컬라이제이션이 왜 필요한가?

- 로봇에게 정확한 지도가 있어도, 지도 위의 자신의 위치를 모르면 무용지물
 - "지도 속 나는 어디에 있나?"
 - 처음 켜진 로봇은 지도상의 자신의 위치(x , y , 방향 θ)를 모를 수 있음
 - 마치 낯선 건물 안에서 평면도 한 장만 들고 서 있는 것과 같음
 - 오도메트리(Odometry)의 한계
 - 바퀴 회전수·IMU로 "내가 얼마나 움직였나"를 계산하는 방식
 - 현실에서는 바퀴 미끄러짐(Wheel Slip), 센서 오차가 섞임
 - 처음엔 1cm 차이였던 오차가 시간이 지나면 수십 cm 이상으로 누적
 - 해결책: 센서 + 지도 비교로 누적 오차를 지속적으로 보정



- 로봇이 처한 상황에 따라 위치 추정의 난이도가 달라짐
 - ① 위치 추적 (Position Tracking) — 가장 쉬운 단계
 - 초기 위치를 정확히 아는 상태에서, 이동하는 로봇의 위치를 지속 업데이트
 - ② 글로벌 로컬라이제이션 (Global Localization) — 중간 난이도
 - 로봇이 지도의 어디에 있는지 **전혀 모르는** 상태에서 시작
 - 지도 전체에 "혹시 여기인가?" 하는 가설을 흘려야 함
 - ③ 납치된 로봇 문제 (Kidnapped Robot Problem) — 가장 어려운 단계
 - 잘 이동 중이던 로봇이 갑자기 **다른 곳으로 옮겨진 상황**
 - 자신이 생각한 위치가 완전히 틀렸음을 인지하고 위치를 처음부터 재탐색

Part 2. 파티클 필터 (Particle Filter)

- "어려운 수학 문제를 풀기 어렵다면, 수많은 가설을 던져서 맞히자"
 - 파티클(Particle, 입자)이란?
 - 지도 위에 흩뿌려진 수많은 '가상 로봇'들
 - 각 파티클은 "로봇이 여기에, 이 방향을 보고 있을 것" 이라는 하나의 가설
 - 가중치(Weight) 부여 과정
 - 로봇이 LiDAR로 주변 벽까지의 거리를 측정
 - 각 파티클 위치에서 "만약 진짜 로봇이 여기 있다면 무엇이 보였을까?"를 계산
 - 실제 센서 값과 비슷하면 높은 가중치, 다르면 낮은 가중치
 - 리샘플링(Resampling) — 생물 진화의 비유
 - 점수 낮은 파티클은 도태, 점수 높은 파티클은 복제·증식
 - "적자생존" 원리 → 가능성 높은 가설에 연산 자원을 집중

- MCL(Monte Carlo Localization): 파티클 필터를 로봇 위치 추정에 적용한 알고리즘
 - ① 예측 단계 (Motion Update)
 - 로봇이 1m 움직이면, 모든 파티클도 각자의 위치에서 1m 이동
 - 약간의 무작위 노이즈를 섞어 파티클을 흩어지게 함
 - → 오도메트리의 불확실성을 반영
 - ② 업데이트 단계 (Sensor Update)
 - 로봇이 LiDAR로 벽을 스캔
 - 지도와 스캔 데이터를 비교하여 각 파티클의 가중치(확률)를 계산
 - ③ 리샘플링 단계 (Resampling)
 - 높은 확률의 파티클들만 남기고, 그 주변에 새 파티클들을 채움
 - 이 루프 반복 → 파티클들이 실제 위치로 수렴(Convergence)

- MCL 루프 흐름

초기화

↓

지도 전체에 파티클을 무작위 배포

↓

[루프 시작]

① 예측: 로봇 이동량만큼 파티클 이동 + 노이즈 추가

↓

② 업데이트: LiDAR 스캔 vs 지도 비교 → 각 파티클 가중치 계산

↓

③ 리샘플링: 가중치 기반으로 파티클 재선택 (높은 확률 복제, 낮은 확률 제거)

[루프 반복]

↓

파티클 수렴 → 로봇 위치 확정

- 파티클 수는 정확도와 연산량의 균형이 중요
 - 파티클이 많을수록
 - 가설이 촘촘하여 위치를 더 정확하고 빠르게 찾음
 - 하지만 CPU가 각 파티클마다 가중치를 계산 → 연산 부담 증가
 - 너무 많으면 로봇의 제어 속도가 느려질 수 있음
 - 파티클이 적을수록
 - 연산량이 적어 빠른 처리 가능
 - 하지만 가설이 듥성듬성하여 위치 추정 정밀도 저하
 - 실용적 기준
 - 일반적으로 500 ~ 2000개 수준에서 운용
 - 로봇의 CPU 성능과 환경 복잡도에 맞게 조정

Part 3. AMCL (Adaptive MCL)

- 기본 MCL은 파티클 수를 고정 / AMCL은 상황에 맞게 파티클 수를 자동 조절
 - KLD-Sampling 개요
 - KLD(Kullback-Leibler Divergence): 추정 분포와 실제 분포의 차이를 측정하는 척도
 - 위치를 확신할 때 (파티클들이 한곳에 수렴)
 - 파티클 수를 줄여서 CPU 자원을 절약
 - 위치를 잃었을 때 (가설이 사방으로 퍼짐)
 - 파티클 수를 대폭 늘려 로봇을 다시 찾을 확률을 높임
 - 결과: 효율성과 생존율을 동시에 확보

- 전체 TF 흐름: `map → odom → base_link`
 - 오도메트리 (`odom → base_link`)
 - 바퀴 회전량 등을 이용해 **연속적이고 빠른** 상대 이동 거리 계산
 - 시간이 지날수록 오차 누적
 - AMCL (`map → odom`)
 - 지도를 기준으로 LiDAR 데이터를 비교
 - 오도메트리의 누적 오차를 보정하는 절대 위치 변환 계산
 - 역할 분담
 - 오도메트리: 로봇의 즉각적인 움직임 담당
 - AMCL: 그 움직임이 지도의 어느 좌표에 해당하는지 주기적으로 보정

A* 이론 & Nav2 플래너 연결

경로 계획 알고리즘

cchyun@gmail.com

Part 1. NavFn 플래너와 Dijkstra 알고리 즘 기초

- 자율주행 내비게이션을 구성하는 두 가지 축

- 글로벌 경로 계획기 (Global Planner)

- 전체 격자 지도(Static Map) 정보를 바탕으로 출발지부터 목적지까지의 거시적이고 안전한 최적의 경로를 계산
(예: 내비게이션 앱의 추천 경로 탐색)

- 로컬 경로 계획기 (Local Planner)

- 로봇이 실제 경로를 주행하면서 마주하는 실시간 동적 장애물을 회피하고 제어 명령을 모터에 전달함
(예: 운전자의 실시간 조향)

- 글로벌 경로 계획기의 필수 충족 조건

- 정적 장애물 및 지형 경계(벽)를 확실하게 회피하여 충돌 없는 경로를 만들어야 함
- 주행 시간과 거리가 최소화되는 비용 효율적(Cost-effective)인 최적 경로를 보장해야 함

- NavFn 플래너의 핵심 특징

- 2D 점유 격자 지도(Occupancy Grid Map)를 기본 공간 표현으로 사용함
- 글로벌 코스트맵(Global Costmap) 상에서 목적지로부터 출발지 방향으로 경로 탐색 비용(Cost Propagation)을 채워 넣음
- 내부 알고리즘 설정에 따라 Dijkstra 또는 A* 알고리즘을 선택하여 최단 비용 경로를 획득함

- 안전한 장애물 회피 방식

- 단순한 이진 충돌 방지를 넘어, 장애물 경계 주변에 **인플레이션 반경(Inflation Radius)** 영역을 설정함
- 벽이나 가구 등 장애물에 극도로 밀착하여 주행하지 않도록 감쇄형 비용 가중치를 부여하여 안전거리를 유지함

- 비용 평가 수식

- A* 알고리즘의 총 비용 평가식 $f(n) = g(n) + h(n)$ 에서 목적지 방향 정보인 $h(n) = 0$ 으로 둔 형태임
- 즉, 현재 노드까지 도달하는 데 소요된 실제 누적 이동 비용($f(n) = g(n)$)만을 반영함

- 맹목적 탐색 방식 (Uninformed Search)

- 목적지의 방향을 모르기 때문에, 인접 노드들을 향해 사방으로 동일하게 확장해 나감
- 뚜렷한 방향 힌트(휴리스틱)가 부재하여 출발점을 기준으로 동심원(또는 격자상 마름모) 형태로 탐색 공간이 비효율적으로 팽창함

Dijkstra 알고리즘: 최저 비용 노드의 점진적 확정 절차

- Dijkstra 알고리즘 탐색 흐름

- Step 1: 출발지의 누적 비용(g)을 0으로 초기화한 뒤 **Open List**에 등록함
- Step 2: Open List 노드 중 누적 비용이 가장 적은 노드를 꺼내 탐색을 완료하고 **Closed List**로 이전함
- Step 3: 확정된 노드의 주변 인접 노드들을 확인하여 각 인접 노드의 새로운 누적 비용(g)을 산출함
- Step 4: 기존에 기록되어 있던 경로보다 적은 누적 비용으로 해당 이웃 노드에 도달할 수 있다면 비용을 업데이트함
- Step 5: 최종 목적지가 Closed List에 등록될 때까지 위의 과정을 계속 반복함

- 최적성 보장

- 비용이 낮은 노드부터 철저히 검증하며 넓히므로, 목적지 도달 시 해당 경로는 수학적으로 최단 경로(Optimal Path)임이 보장됨

-
- **Open List**: 탐색 후보군으로 등록되었으나 주변 노드 확장이 아직 진행되지 않은 상태 목록
 - **Closed List**: 탐색을 마쳐 최단 도달 거리가 영구적으로 확정된 노드 목록

- 치명적인 단점

- 목적지가 특정 방향에 고정되어 있음에도 불구하고, 반대 방향을 포함한 모든 방향을 골고루 넓혀가며 탐색함 (눈먼 탐색)
- 맵의 해상도가 높아지거나 지도가 광활해질수록 연산 대상 노드가 폭발적으로 증가하여 실시간 연산 성능이 크게 급감함

- 지능형 탐색으로의 전환

- "목적지의 방향을 대략 알고 있다면 그 방향 위주로 먼저 뺀어 나가며 탐색하는 것이 합리적이지 않은가?"
- 이 직관을 반영하여 실제 누적 비용(g)에 목적지까지의 추정 비용(휴리스틱, h)을 결합한 것이 **A* 알고리즘**

Part 2. A* 알고리즘 직관적 이해

글로벌 경로 계획(Global Path Planning)이 필요한 이유

- 로봇이 장애물을 회피하여 최적의 순서로 목적지에 안착하게 만드는 핵심 거동 설계 과정
 - 물리적 세계의 제약 사항
 - 현실 세계의 실물 로봇은 단순한 기하학적 궤적으로만 움직이지 못하며, 지도상에 벽, 가구, 단차 등 통행 불가 구역이 상존함
 - 안전 반경을 확보하기 위해 로봇의 형상 정보(Footprint)를 코스트맵에 반영하고 장애물과의 이격 거리를 계산해야 함
 - 단순 근시안적 주행 알고리즘의 한계
 - 무작위 주행 (Random Walk): 목적지 도달이 운에 따라 결정되므로 주행 시간이 무한정 늘어날 수 있음
 - 버그 알고리즘 (Bug Algorithm): 벽을 따라 무작위 회피를 하는 단순 기하 제어는 'ㄷ'자 등의 막다른 골목 (Local Minima)에 갇히면 탈출이 원천적으로 불가능함
 - 글로벌 경로 계획의 당위성
 - 맵 전역의 정보를 사전에 파악하여 막다른 구역을 원천 차단하고, 전체 탐색 비용을 고려한 장기적 관점의 정밀한 경로를 도출해야 함

- 로봇은 물리 세계의 연속 공간을 노드(Node)와 엣지(Edge)의 이산적 그래프 구조로 재구성함
 - 격자형 지도 표현 (Occupancy Grid Map)
 - 공간을 격자망으로 잘게 나누어 노드화함 (예: 5cm 격자 단위 = 1노드)
 - 빈 격자 칸의 인접 노드 간 연결 선로를 **엣지(Edge)**로 설정하고, 장애물이 채워진 격자 노드는 엣지를 제거함
 - 제어 탐색 알고리즘 비교
 - Dijkstra: 누적 주행 거리(g)만을 기준으로 가중치를 늘려 최적성은 지키나 목적지 위치를 고려하지 않아 전 방향으로 팽창하는 맹목적 탐색(Uninformed Search)을 수행함
 - A*: 누적 주행 비용(g)에 목적지 방향의 예상 비용(휴리스틱, h) 힌트를 더해 탐색 흐름을 목적지 쪽으로 강력하게 편향시키는 지능형 탐색(Informed Search)을 수행함

A* 알고리즘 핵심 수식: $f(n) = g(n) + h(n)$

- 과거의 실측 비용과 미래의 예측 비용을 유기적으로 합산하여 노드의 유망성을 평가함

$$f(n) = g(n) + h(n)$$

- 각 평가 항의 수학적 의미와 성질

- $g(n)$ (출발지로부터의 실적 비용): 시작 노드부터 현재 탐색 중인 노드 n 까지 도달하는 데 소요된 실제 누적 이동 거리 혹은 누적 코스트값 (예: 가로/세로 이동 10, 대각선 이동 14)
- $h(n)$ (목적지까지의 예측 비용, Heuristic): 현재 노드 n 에서 목표점까지 남은 잔여 경로에 대한 낙관적 추정치 (장애물이 전혀 없는 빈 평면상의 가상 직선거리로 계산)
- $f(n)$ (총합 평가 지수): $f(n)$ 수치가 가장 낮은 노드 순으로 우선 탐색하며, "이 경로로 경유할 때 최종 누적 거리가 가장 짧아질 것"임을 의미함

- 연산 자원 최소화를 위해 두 개의 노드 관리 리스트를 운용함
 - Open List
 - 탐색 과정에서 발견되었으나 주변 인접 영역으로 확장이 아직 실행되지 않은 후보 노드군
 - 리스트 내부에서 총합 평가 지수 $f(n)$ 이 가장 작은 노드가 맨 앞으로 오도록 정렬하여 탐색 대상으로 선정함
 - Closed List
 - 이미 주변 인접 노드로의 확장을 모두 마치고 최단거리가 확정된 정적 노드군
 - 한 번 방문이 끝난 노드의 중복 재탐색 및 무한 루프 진입을 방지하는 방어벽 역할을 수행함

- 목적지 거리 추정 함수 $h(n)$ 의 기하학적 형태에 따라 탐색 효율이 직접 결정됨
 - 맨해튼 거리 (Manhattan Distance)
 - 상하좌우 격자 축(4방향)으로만 로봇이 움직일 수 있는 이산 이동 제약 환경에 적합함
 - $h(n) = |x_1 - x_2| + |y_1 - y_2|$
 - 유클리드 거리 (Euclidean Distance)
 - 대각선을 포함하여 모든 방향으로 장애물이 없는 빈 공간을 직접 활공하는 다방향 연속 제어계에 최적화됨
 - $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$
 - 허용성 조건 (Admissible Heuristic)
 - **중요 규칙:** 휴리스틱 함수 $h(n)$ 은 실제 최종 목적지까지 남은 실제 거리보다 크게 추정해서는 안 됨 (과대평가 금지 법칙)
 - 항상 실제 경로비보다 작거나 같게 낙관적으로 예측해야만, 알고리즘이 발견되지 않은 최적의 지름길을 거르지 않고 완벽한 최단 경로를 보증해 냄

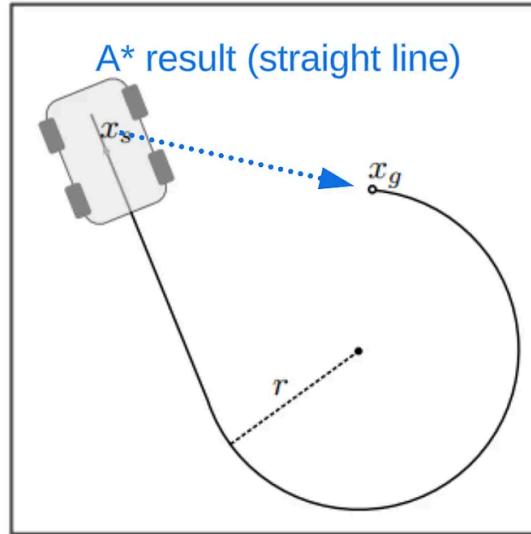
- 2D 점유 격자 맵 환경에서 가장 독보적이며 널리 입증된 고속 경로 계획법
 - 확실한 강점
 - 2차원 평면 격자 맵 상에서 모바일 플랫폼의 실시간 경로 생성이 가능할 정도로 대단히 빠르고 가벼움
 - 허용성 조건(Admissibility)을 만족하는 휴리스틱 사용 시, 수학적으로 반드시 완벽한 최단 경로 검출을 보장함 (Completeness & Optimality)
 - 치명적 한계: 차원의 저주 (Curse of Dimensionality)
 - 로봇의 주행 방향(θ)을 가산한 3차원 공간 (x, y, θ) 제어나 다관절 로봇 팔의 결합 상태 등 고차원(자유도) 공간 탐색 시 격자의 수가 기하급수적으로 폭발함
 - 연산 자원의 고갈로 탐색이 불가능해지므로, 연속 공간을 임의로 샘플링하여 엮는 **RRT 계열 알고리즘**으로 선회해야 함

Part 3. Smac Planner 직관적 이해

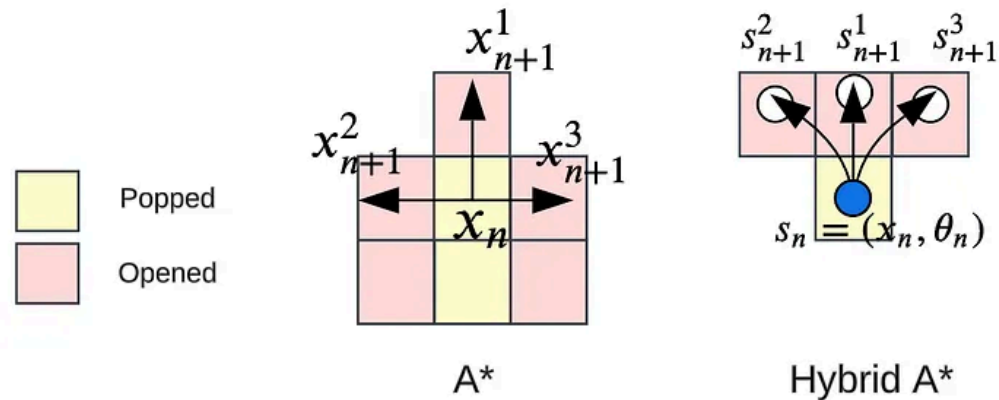
- 기존 글로벌 플래너(NavFn)의 이론적 한계를 보완하기 위해 개발된 고성능 전역 경로 계획 솔루션
 - 등장 배경 및 필요성
 - 기존 2D 격자 기반 A*는 로봇을 하나의 점(Point)으로 삭감하여, 로봇의 주행 각도(Heading)나 회전 반경을 무시한 채 꺾임이 심한 지그재그(Zig-zag) 경로를 생성함
 - 조향 시스템 구조상 제자리 회전이 힘든 아커만(Ackermann) 및 차량형 주행 로봇은 실제 선회 능력이 반영된 현실적인 기하 곡선 경로가 절대적으로 필요함
 - Smac Planner의 3대 라인업
 - **Smac 2D**: 기존 A* 탐색을 극한으로 최적화하여 대형 맵에서도 초고속 반응 성능을 구현한 2D 격자 플래너
 - **Smac Hybrid-A***: 회전 각도(θ) 및 선회 곡률 한계를 고려하여 차가 실제로 주행 가능한 스무드한 조향 경로를 산출하는 플래너
 - **Smac State Lattice**: 차량의 최대 가속 한계와 기구 동역학 프로파일을 고려하여 정교한 조종 궤적을 엮어내는 최고급 플래너

- 표준 A*의 메모리 접근 및 탐색 구조를 리팩토링하여 연산 부하를 획기적으로 차단함
 - 코스트맵 다운샘플링 (Costmap Downsampling)
 - 5cm 미만의 고해상도 지도를 날것 그대로 연산하면 연산 자원이 고갈됨
 - 탐색 초기에는 해상도를 한시적으로 조절하여 대략적인 경로 뼈대를 빠른 속도로 찾고, 최종 세부 단계에서 해상도를 복원하여 최적화함
 - 메모리 사전 할당 구조 (Memory Pre-allocation)
 - C++ 런타임 탐색 중 지속적인 노드 메모리 동적 할당/해제는 메모리 파편화 및 성능 저하를 일으킴
 - 구동 시점에 예측 가능한 최대 노드 크기만큼 메모리 풀(Pre-allocated Node Pool)을 잡아두어 C++ 연산 처리 속도를 최고 수준으로 끌어올림
 - 비용 인지형 탐색 (Cost-aware Search)
 - 장애물 충돌 방지 외에도 인플레이션에 따른 장애물 근접 위험 누적 가중치 비용을 $f(n)$ 에 적극 반영함
 - 거리가 조금 우회하더라도 로봇이 좁은 통로의 중앙선이나 넓고 안전한 중앙 광장 위주로 유도되게 설계됨

- 상태 차원의 확장: 2D 평면 위치 정보 (x, y) 에서 방향각을 더한 3D 상태 공간 (x, y, θ) 탐색
- 격자와 연속 기하 공간의 하이브리드 결합
 - 일반적인 격자형 A*는 무조건 인접 격자 칸의 정중앙 정수형 좌표로만 노드를 확장해 나감
 - Hybrid-A*는 로봇의 실제 바퀴 조향각을 시뮬레이션하여 뺀 연속 좌표 (x, y, θ) 를 노드로 삼아, 물리적으로 주행할 수 있는 정교한 곡선을 그려냄



- 로봇의 회전 제약 한계를 탐색 트리에 강제하여 주행 불가능한 경로 생성을 원천 배제함
- 비홀로노믹 운동학 제약 (Non-holonomic Constraints)
 - 자동차와 같이 옆으로 게걸음할 수 없는 기구적 한계를 제어 수식에 강제 반영함
 - 로봇의 최소 회전 반경(Minimum Turning Radius)을 매개변수화하여, 물리적으로 꺾을 수 없는 예리한 각도의 코너링 궤적은 후보군에서 원천 제외함



- 차량의 관성, 가속 한계 등 복합 동역학(Kinodynamic) 사양을 설계에 반영하는 고정밀 제어형 플래너
 - 모션 프리미티브 (Motion Primitives) 활용
 - 런타임 제어 주기마다 복잡한 미분 곡선을 계산하는 방식 대신, 로봇 기구가 물리적으로 실행할 수 있는 극소 단위의 스무드한 궤적 조각(Motion Primitive)들을 사전에 정밀하게 생성하여 라이브러리화해 둠
 - 탐색 시 이 궤적 조각들을 격자망(Lattice Grid) 위에 레고 블록을 맞추듯 정밀하게 결합해 목적지 경로를 도출함
 - Hybrid-A* 플래너와의 차별성
 - Hybrid-A*: 아커만 조향 로봇의 기하학적 궤적과 조향각 한계 등 운동학에 주로 초점을 맞춤
 - State Lattice: 전방향 구동 휠(Omnidirectional), 메카넘 휠, 특수 트랙터 등 기구적 특성이 다른 모든 로봇의 속도 가감속 특성까지 경로 궤적에 그대로 녹여내어 하드웨어 범용성이 매우 뛰어남

- Nav2는 플러그인 아키텍처를 도입하여 하드웨어 특성에 적합한 글로벌 경로 계획기를 바인딩함
 - NavFn 플래너 (Dijkstra/A* 기반)
 - 2D 평면 격자 상에서 시작점부터 목표점까지의 기하 최단거리를 도출함
 - 제자리 선회(Spin-in-place)가 가능하여 방향각의 제약이 적은 원형 차동 구동(Differential) 로봇에 적합함
 - Smac Planner Hybrid (Hybrid-A* 기반)
 - 일반 A*에 로봇의 최소 회전 반경 제약 수식을 결합함
 - 좁은 코너에서 자동차 방식(Ackermann)의 차량 로봇이 불가능한 궤적으로 꺾지 않도록 구동 한계를 강제함
 - Smac Planner Lattice (State Lattice 기반)
 - 사전에 검증된 로봇 전용 모션 프리미티브 조합망을 활용해 경로를 엮어냄
 - 전방향 휠 및 비정형 조향 장치를 갖추어 전용 물리 궤적 모델을 필수적으로 따라야 하는 특수 플랫폼에 적합함

- 이산 격자 지도 기반 플래너가 도출하는 계단형(Zig-zag) 경로의 한계 보완
 - 직각 경로의 계단 현상 완화
 - A* 등 격자 플래너가 생성한 경로는 격자를 따라가기 때문에 직각 형태임
 - 로봇이 해당 경로를 추종하면 꺾이는 매 지점마다 회전을 위해 감속/가속을 반복해야 하므로 주행 효율이 심각하게 저하됨
 - Nav2 SmootherServer의 작동
 - 뼈대 역할을 하는 글로벌 경로를 전처리 입력으로 전달받아 곡선 곡률을 다듬음
 - 로봇의 회전 선회력 한계를 훼손하지 않으면서 부드러운 스플라인 곡선 경로로 평활화함



- 로봇 하드웨어 사양 및 주행 구동 설계 특성에 따른 최적의 플러그인 선택 기준
 - NavFn 플래너를 선정해야 하는 경우
 - 차동 주행 및 옴니휠 로봇과 같이 제자리 회전 반경 제약이 없는 플랫폼
 - 하드웨어 리소스 제약이 크고, 연산 오버헤드 없이 신속하게 2D 최단 경로를 얻고 싶을 때
 - Smac Hybrid-A* 플래너를 선정해야 하는 경우
 - 자동차 조향형(Ackermann) 플랫폼 및 최소 회전 반경이 큰 대형 물류 AGV
 - 벽면이나 장애물에 충돌하지 않고 꺾을 수 있는 주행 가능한 선회 경로를 도출해야 할 때
 - Smac Lattice를 선정해야 하는 경우
 - 조향 모터에 복잡한 가감속 제어 한계(Dynamic Constraint)가 강제되는 특수 조향 플랫폼
 - RRT* 알고리즘을 선정해야 하는 경우
 - 모바일 주행을 넘어 6자유도 이상의 매니퓰레이터 로봇 팔 동작 및 충돌 회피 계획 수립 시